Git Commit

Introduction

Imagine you are working on a **coding project**, and you have made **several changes** to your files. You want to **save these changes in Git** so that you can track them, share them, or even undo them later.

How do you save changes in Git permanently?

This is where the git commit command comes in!

Section 1: Learn

What is git commit?

git commit **is a Git command used to save changes in the repository.** It creates a snapshot of the project, preserving the current state of all files that were staged.

Why Do We Need git commit?

Without Git (git commit**)**	**With Git (**git commit**)**
Changes are lost after closing the editor	Changes are permanently saved
No history of modifications	Every change is recorded
Difficult to track who made changes	Shows author, date, and message
Cannot restore older versions	Can revert back to any previous commit

How Does git commit Work?

1. **Stage the changes** – Select the files you want to commit using git add.

   2. **Commit the changes** – Save them in Git using git commit -m "Message".

   3. **Push to Remote** – Upload your commits to GitHub or another remote repository.

An Interesting Anecdote

Before Git, developers manually **saved backup files** like project_v1, project_v2, final_project. This caused **confusion and lost work**. Git solved this problem by **allowing structured commits with messages**, making project management efficient.

Section 2: Practice

Step-by-Step Guide to Using git commit

1. Setting Up a Git Repository

```
# Create a new project folder
mkdir my_project
cd my_project

# Initialize Git in the folder
git init
```

Why is this important?

- Starts version control for your project.
- Creates a hidden .git folder to track changes.

---

2. Creating and Staging a File

```
# Create a new file
echo "print('Hello, Git!')" > script.py

# Stage the file for commit
git add script.py
```

What does git add **do?**

- Prepares the file to be committed.
- Moves the file from **untracked** to **staged** status.

---

3. Committing Changes

```
# Commit the file with a message
git commit -m "Added script.py file"
```

What happens here?

- Git saves the **current state** of the project.
- The commit message **describes the change**.

---

4. Checking Commit History

```
# View commit history
git log
```

Expected Output:

```
commit 3a6f8f1c7e2f (HEAD -> main)
Author: Your Name <your.email@example.com>
Date:   Mon Feb 26 10:00:00 2025 +0000

    Added script.py file
```

Why is this useful?

- Shows **who made changes and when**.
- Helps **track project progress over time**.

---

5. Making More Changes and Committing Again

```
# Modify the script.py file
echo "print('Welcome to Git!')" >> script.py

# Stage the updated file
git add script.py

# Commit the changes with a new message
git commit -m "Updated script.py with welcome message"
```

Why do we commit multiple times?

- Allows us to **track different versions** of the file.
- Helps **revert to previous states if needed**.

---

6. Undoing a Commit (Rollback)

```
# Undo the last commit but keep changes
git reset --soft HEAD~1

# Undo the last commit and remove changes
git reset --hard HEAD~1
```

Why is this important?

- Prevents **accidental data loss**.
- Allows **rolling back to stable versions**.

Real-World Example

A **team of developers** is working on a **mobile banking app**. Each member works on different features like **login, payments, and notifications**.

Without Git Commit:

- Changes are **not saved properly**.
- Debugging is **difficult** without history.

With Git Commit:

1. Each developer **commits their changes** separately.
2. Every commit **has a message describing the update**.
3. If a bug appears, they **roll back to a previous commit**.

Result? Efficient teamwork, **better debugging, and structured progress tracking**.

Section 3: Know More

Frequently Asked Questions

1. **What happens if I forget the commit message?**

   Use:

```
git commit
```

2. This opens a text editor where you can write a commit message.
3. **How do I change the last commit message?**

```
git commit --amend -m "New message"
```

4. This updates the last commit message.
5. **What is the difference between** git add **and** git commit**?**
   - git add **stages changes** but does not save them permanently.
   - git commit **saves changes permanently** in Git history.
6. **Can I commit without adding files?**

   No. You must first use git add before running git commit.
7. **Where can I practice Git commits?**
   - Create a **GitHub account** and start a project.
   - Use **GitHub Desktop** for a graphical interface.

Conclusion

The git commit command is **one of the most important Git operations**. It allows developers to **save, track, and manage changes efficiently**.

Start by **staging files, committing changes, and checking history**. Soon, you'll be **managing real-world projects like a pro!**